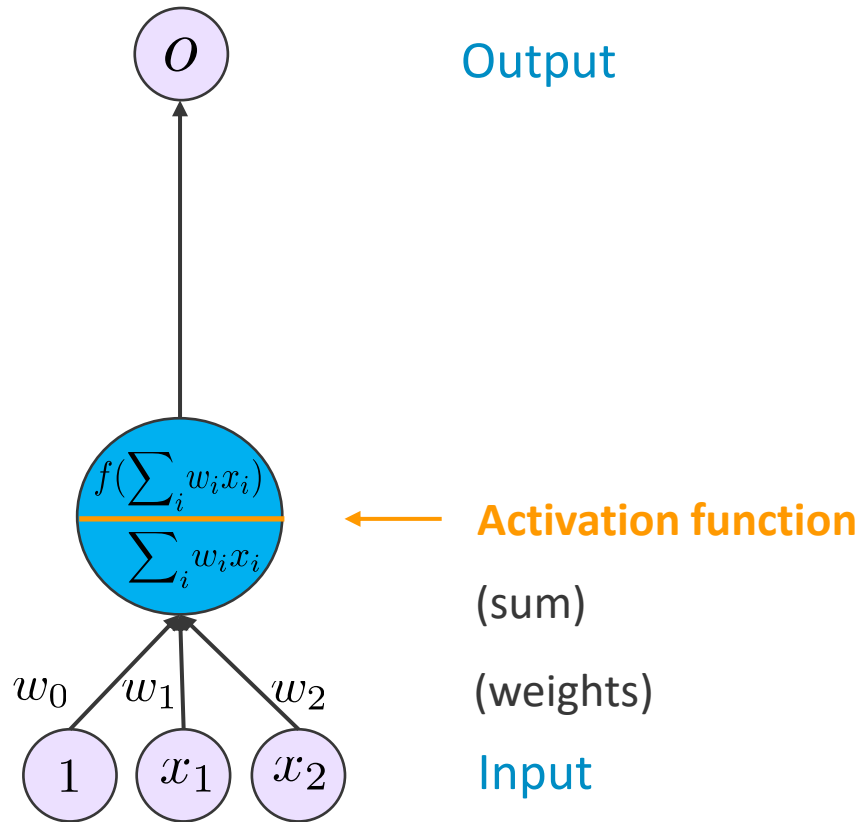


Recurrent Neural Network (LSTM , GRU)

Perceptron (Rosenblatt, 1957)

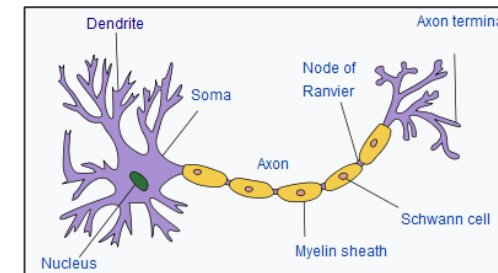


Perceptron*: Given $\{x\}$, predict y
where $y \in \{-1, +1\}$

$$y = f(w_0 + w_1 x_1 + \dots + w_q x_q)$$

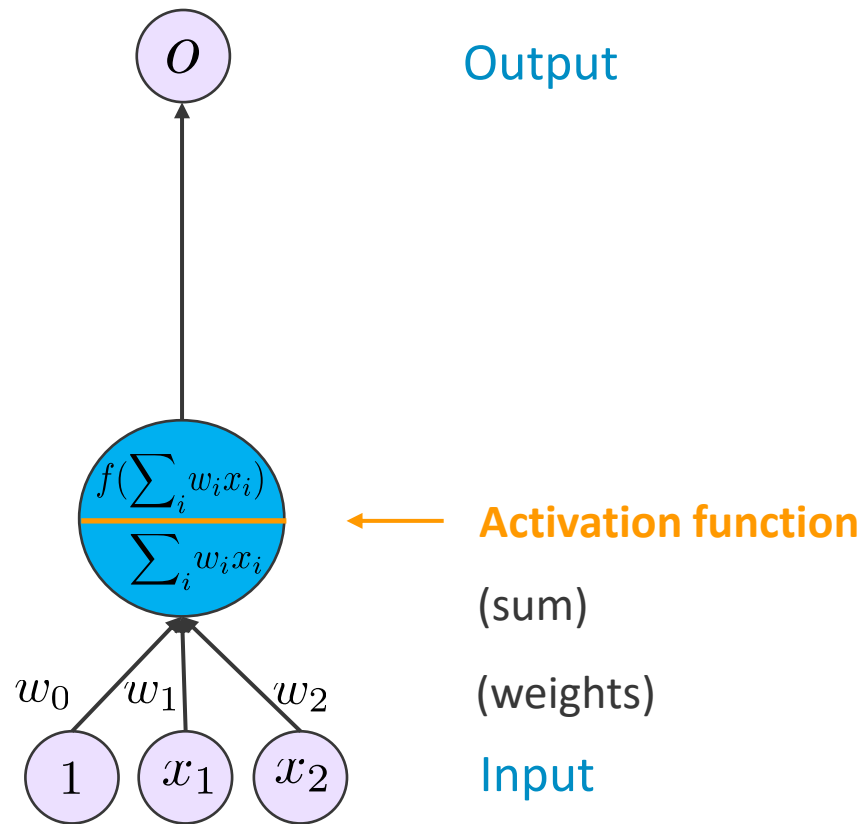
where f is the **step function**:

$$f(x) = \begin{cases} +1, & \text{if } x \geq 0 \\ -1, & \text{if } x < 0 \end{cases}$$



* Non-linear activation function / binary classifier

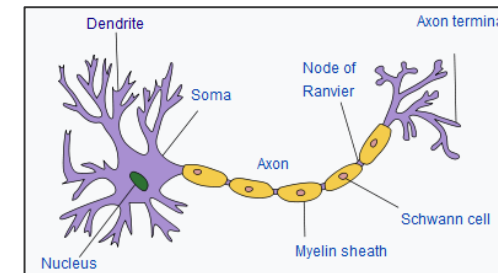
Artificial Neuron



Artificial Neuron*: Given $\{x\}$,
predict y :

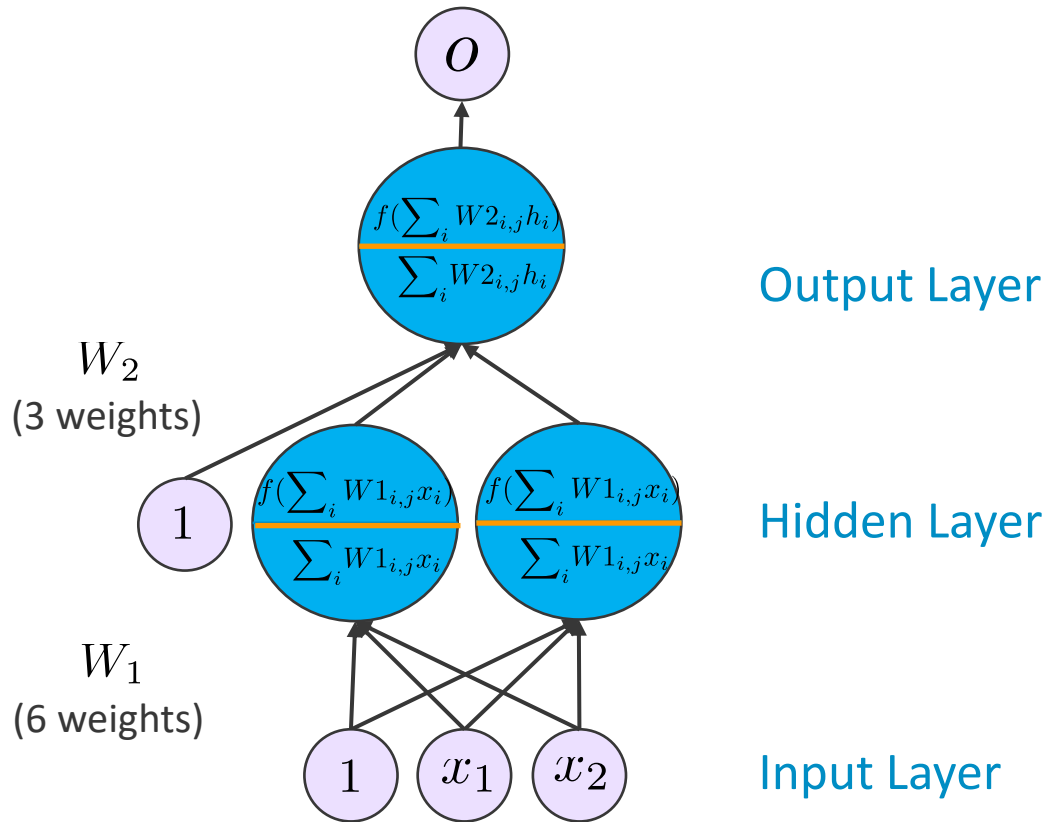
$$y = f(w_0 + w_1x_1 + \dots + w_qx_q)$$

where f is a **nonlinear activation function** (sigmoid, tanh, ReLU, ...)



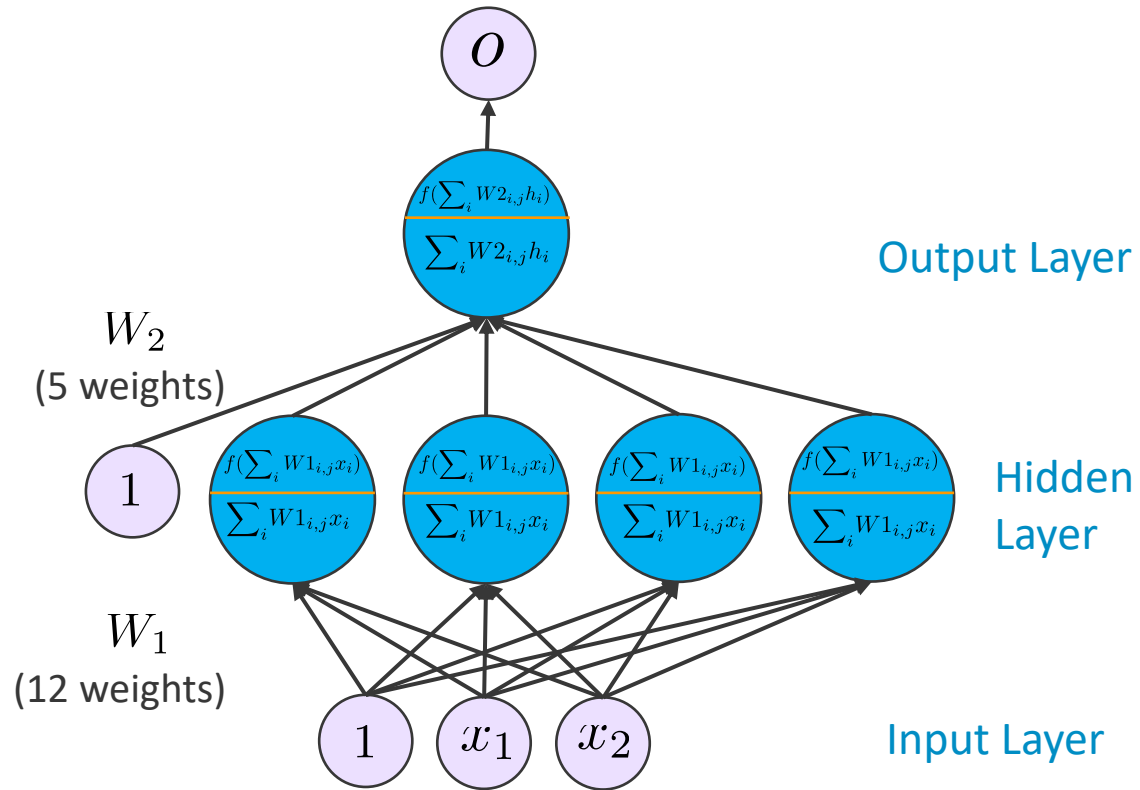
* Similar to how neurons in the brain function

Neural Network/Multilayer Perceptron



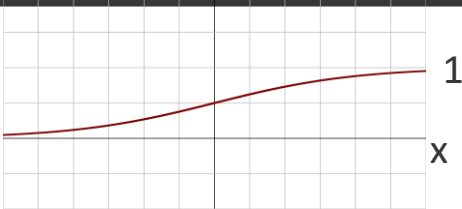
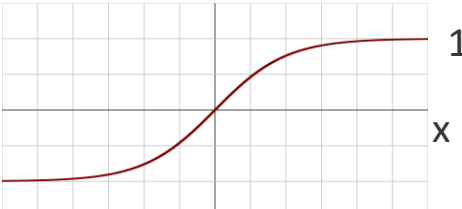
- A neural network consisting of **input**, **hidden** and **output** layers.
- Each layer is connected to the next layer.
- An **activation function** is applied on each hidden layer (and output layer).

Neural Network/Multilayer Perceptron



- A neural network consisting of **input**, **hidden** and **output** layers.
- Each layer is connected to the next layer.
- An **activation function** is applied on each hidden layer (and output layer).

Activation Functions

Name	Plot	Function	Description
Logistic (sigmoid)		$f(x) = \frac{1}{1 + e^{-x}}$	The most common activation function. Squashes input to (0,1).
Hyperbolic tangent (tanh)		$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Squashes input to (-1, 1).
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$	Popular activation function. Anything less than 0, results in zero activation.

Derivatives of these functions are also important (gradient descent).

Output Activations/Functions

- “How to output/predict a result”

Problem	Description	Name	Function
Binary classification	• Output probability for each class, in (0,1) • Logistic regression of output of last layer	Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$
Multi-class classification	• Output probability for each class, in (0,1) • Sum of outputs to be 1 (probability distribution) • Training drives target class values up, others down	Softmax	$f(x_i) = \frac{\exp(x_i)}{\sum_i \exp(x_i)}$
Regression		Linear/ ReLU	$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$

Cost Functions

- “How to compare the outputs with the truth?”

Problem	Name	Function	Notes
Binary classification	Cross entropy for logistic	$C = -\frac{1}{n} \sum_{examples} y \ln(p) + (1 - y) \ln(1 - p)$	Notations for Classification • n = training examples • i = classes • p = prediction (probability) • y = true class (1/yes, 0/no)
Multi-class classification	Cross entropy for Softmax	$C = -\frac{1}{n} \sum_{examples} \sum_{classes} y_i \ln(p_i)$	
Regression	Mean Squared Error	$C = \frac{1}{n} \sum_{examples} (y - p)^2$	Notations for Regression • n = training examples • p = prediction (numeric, $\hat{y}$) • y = true value

Training Neural Networks

- Cost function is selected according to problem: **Binary, Multi-class Classification or Regression.**
- Update network weights by applying **the gradient descent method** and **backpropagation.**

- Weight update formula:

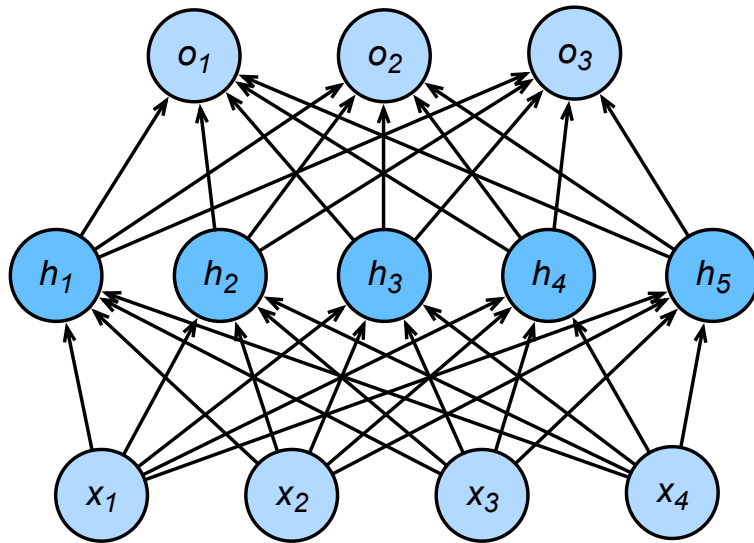
$$w_{new} = w_{old} - learning_rate * \frac{\partial C}{\partial w}$$

C : Cost
Gradient with respect to w

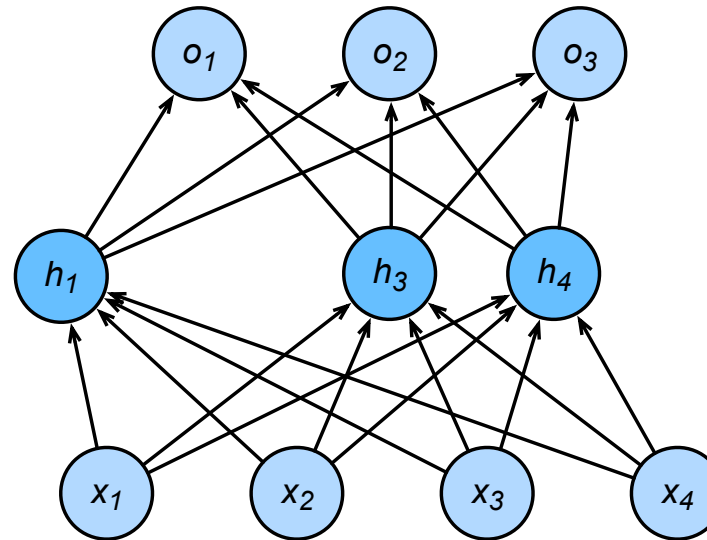
Dropout

- Regularization technique to **prevent overfitting**.
- Randomly removes some nodes with a fixed probability during the training.

MLP with one hidden layer



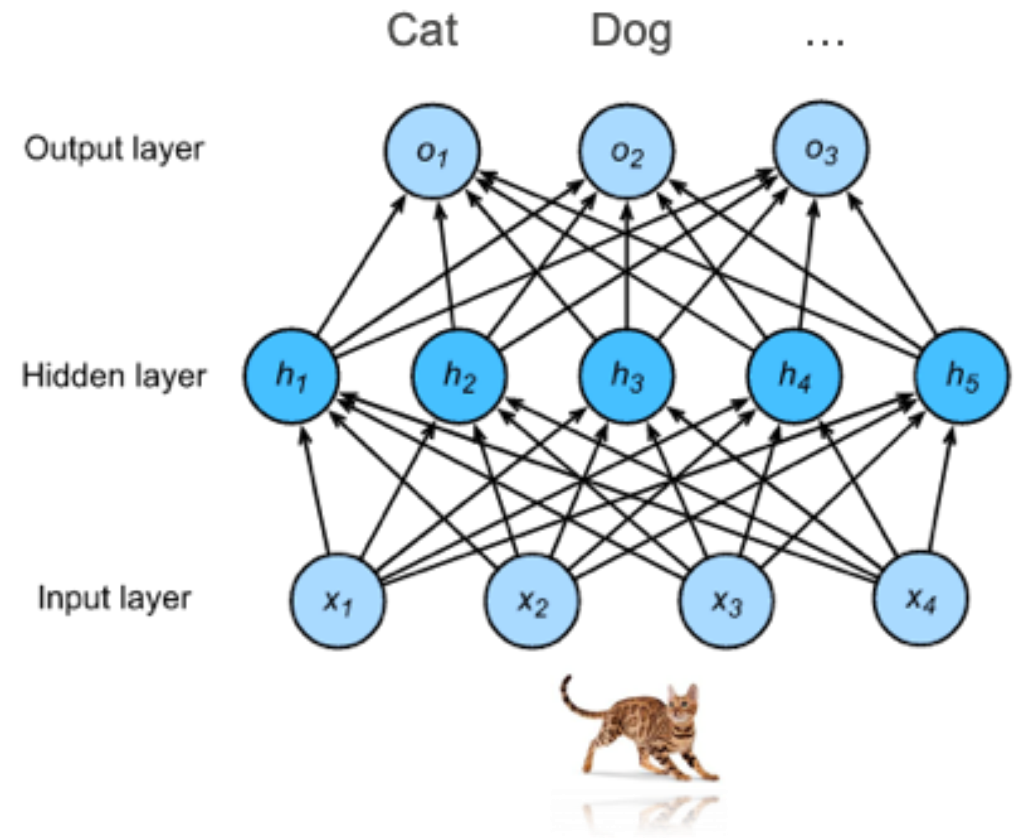
Hidden layer after dropout



[More details](#)

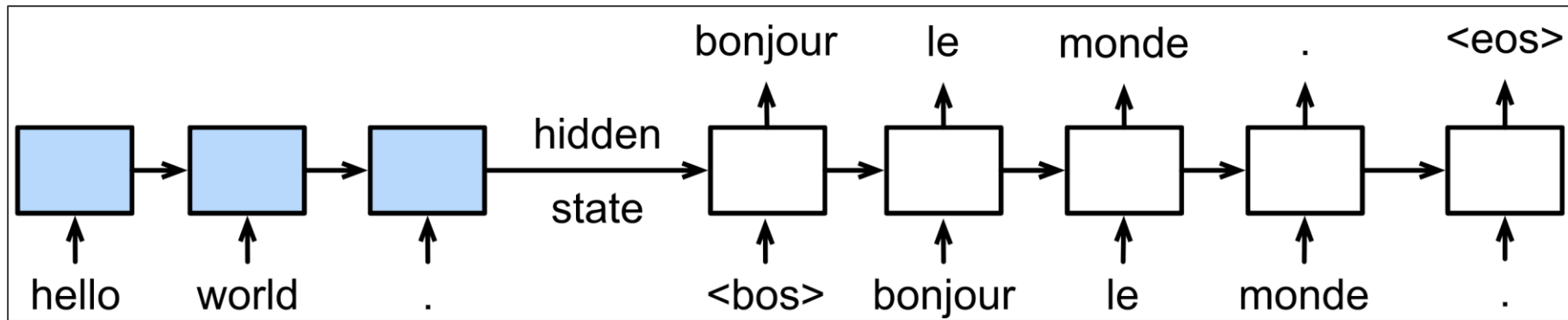
Why Neural Networks?

- Automatically **extract useful features** from input data.
- In recent years, deep learning has achieved **state-of-the art results** in many machine learning areas.
- Three pillars of deep learning:
 - Data
 - Compute
 - Algorithms



Neural Network for Sequential Data?

Text data has sequential information of words.

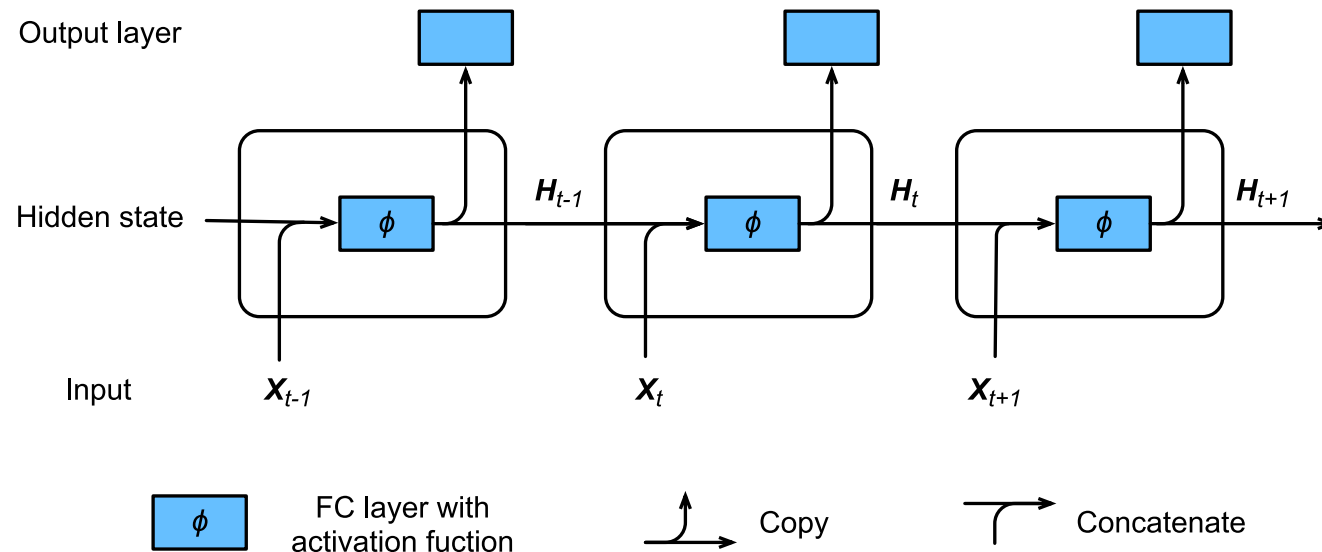


Example: Language translation

Other sequential inputs (time series, music notes, video, etc.)

Recurrent Neural Networks (RNN)

RNN uses an **internal state** to preserve the sequential information between input elements.

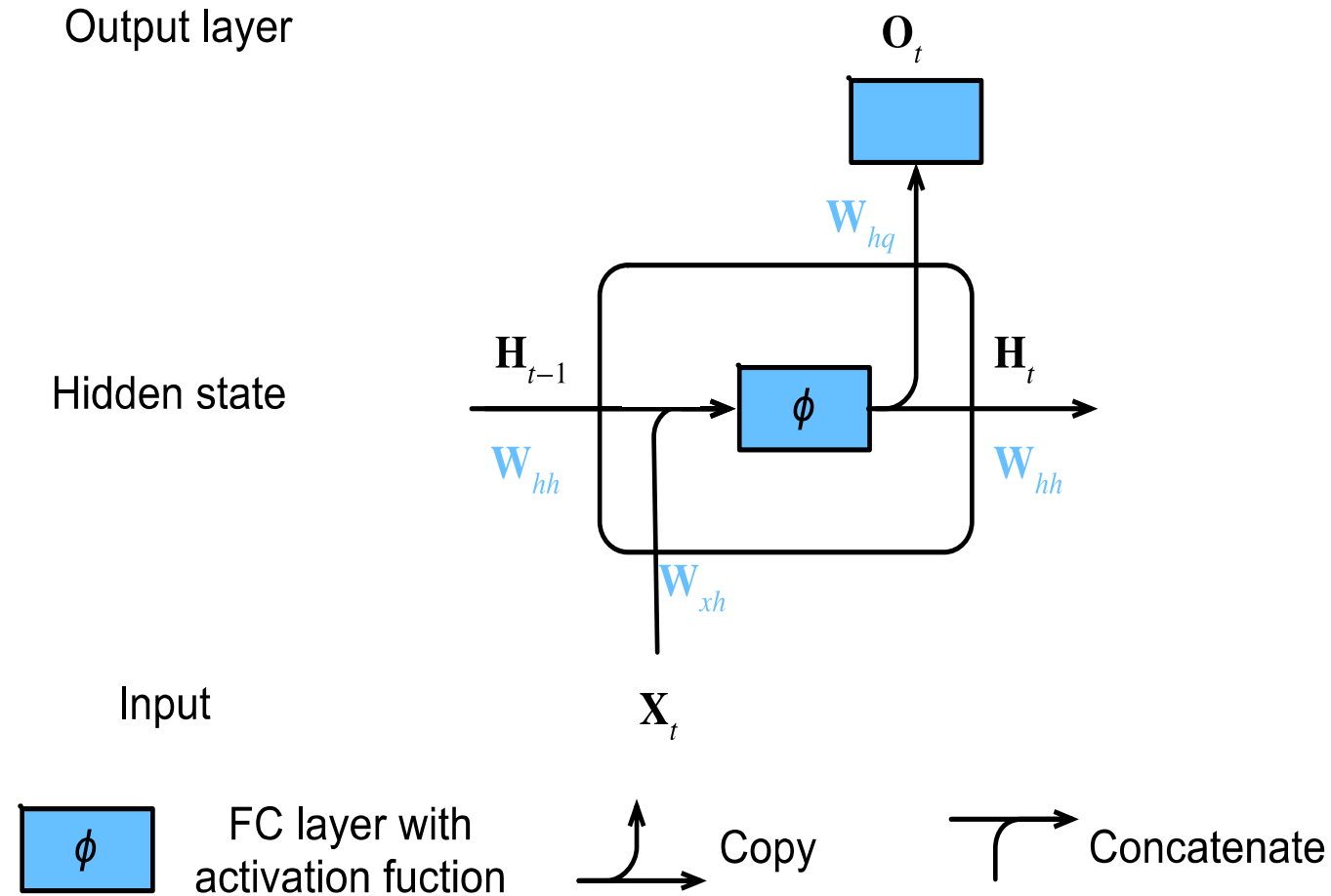


X: Input

H: Hidden state

t: Timestep

The RNN Unit



W_{xh} : Input weight matrix

W_{hh} : State weight matrix

W_{hq} : Output weight matrix

Hidden state update:

$$H_t = \phi(H_{t-1}W_{hh} + X_tW_{xh} + b_h)$$

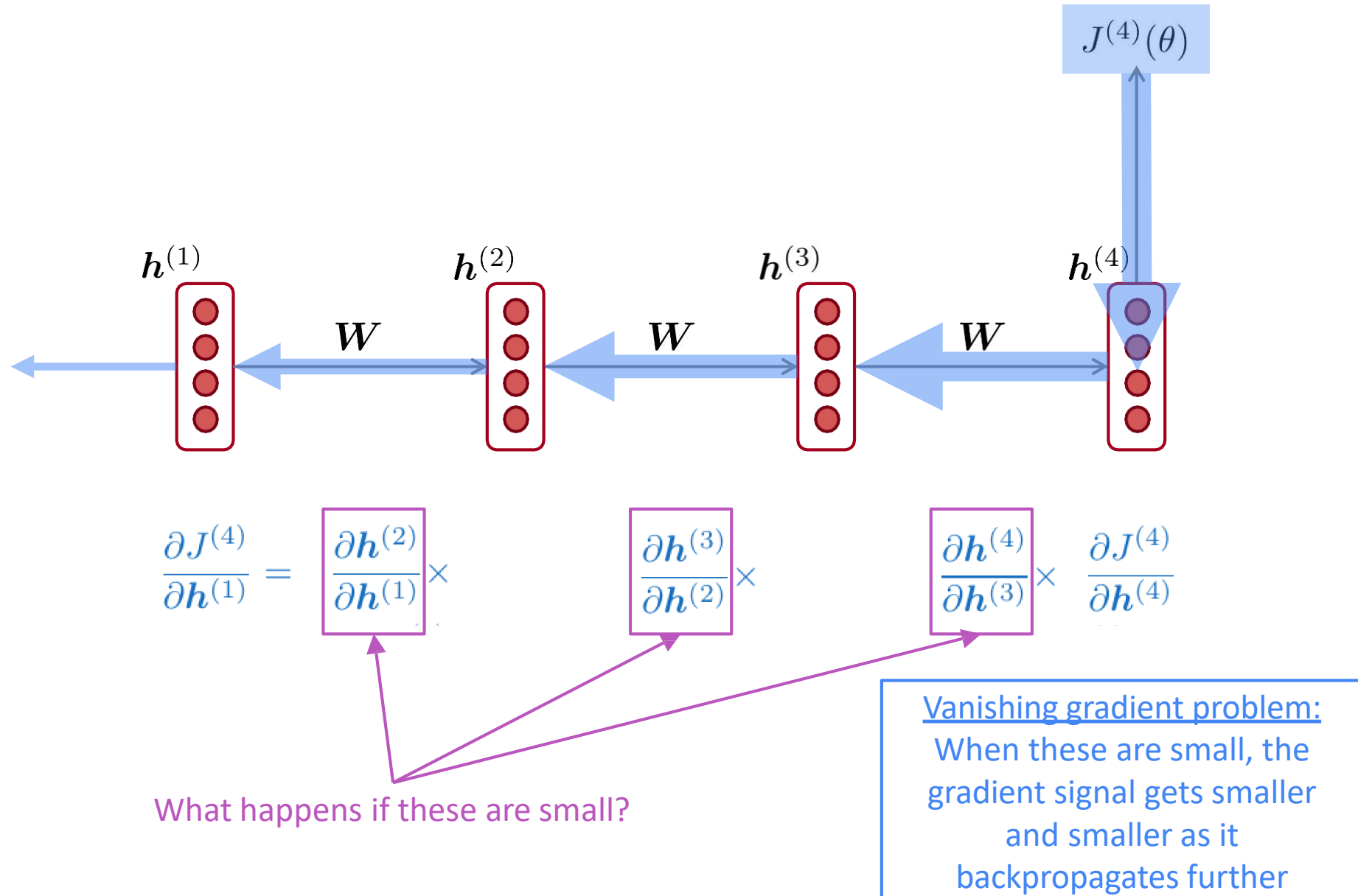
Observation update:

$$o_t = H_tW_{hq} + b_q$$

Effect of vanishing gradient on RNN-LM

- **task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*
- To learn from this training example, the RNN needs to model the dependency between “tickets” on the 7th step and the target word “tickets” at the end.
- But if gradient is small, the model can’t learn this dependency
 - So the model is unable to predict similar long-distance dependencies at test time

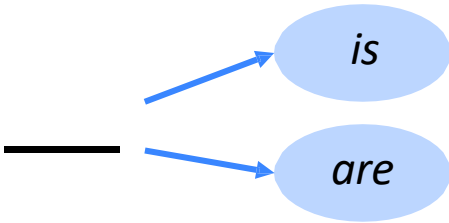
Vanishing gradient intuition



Why is vanishing gradient a problem?

- Another explanation: Gradient can be viewed as a measure of *the effect of the past on the future*
- If the gradient becomes vanishingly small over longer distances (step t to step $t+n$), then we can't tell whether:
 - There's no dependency between step t and $t+n$ in the data
 - We have wrong parameters to capture the true dependency between t and $t+n$

Effect of vanishing gradient on RNN

- **LM task:** *The writer of the books _____*
- **Correct answer:** *The writer of the books is planning a sequel*
- **Syntactic recency:** *The writer of the books is* (correct)
- **Sequential recency:** *The writer of the books are* (incorrect)
- Due to vanishing gradient, RNN are better at learning from sequential recency than syntactic recency, so they make this type of error more often

Gating Hidden States

- Regular RNNs may have difficulty with long-term dependency in input sequences:
 - Not all observations are equally relevant



- Only remember the relevant ones
 - Need mechanism to **pay attention (update gate)**
 - Need mechanism to **forget (reset gate)**

Gated Recurrent Units (GRU)

- Proposed by Cho et al. in 2014 as a simpler alternative to the LSTM.
- On each timestep t we have input $\mathbf{x}^{(t)}$ and hidden state $\mathbf{h}^{(t)}$ (no cell state).

Update gate: controls what parts of hidden state are updated vs preserved

$$\mathbf{u}^{(t)} = \sigma \left(\mathbf{W}_u \mathbf{h}^{(t-1)} + \mathbf{U}_u \mathbf{x}^{(t)} + \mathbf{b}_u \right)$$

Reset gate: controls what parts of previous hidden state are used to compute new content

$$\mathbf{r}^{(t)} = \sigma \left(\mathbf{W}_r \mathbf{h}^{(t-1)} + \mathbf{U}_r \mathbf{x}^{(t)} + \mathbf{b}_r \right)$$

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

$$\tilde{\mathbf{h}}^{(t)} = \tanh \left(\mathbf{W}_h (\mathbf{r}^{(t)} \circ \mathbf{h}^{(t-1)}) + \mathbf{U}_h \mathbf{x}^{(t)} + \mathbf{b}_h \right)$$

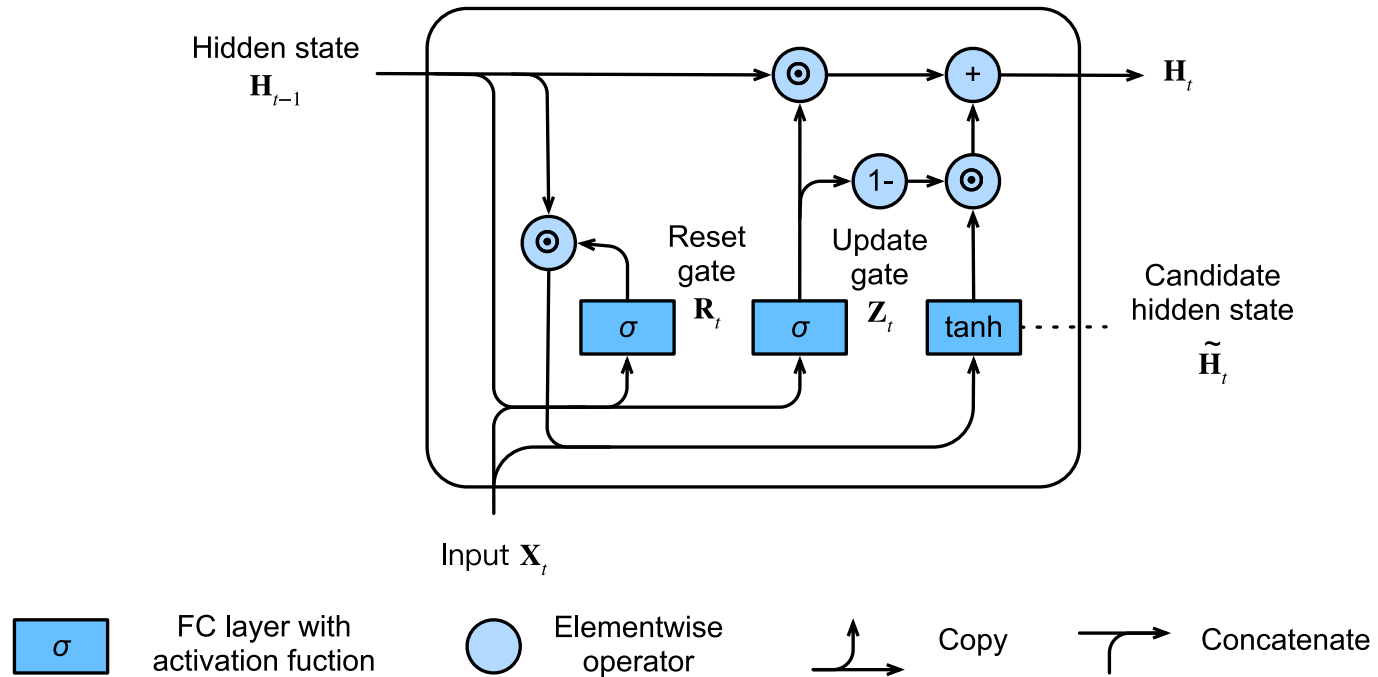
$$\mathbf{h}^{(t)} = (1 - \mathbf{u}^{(t)}) \circ \mathbf{h}^{(t-1)} + \mathbf{u}^{(t)} \circ \tilde{\mathbf{h}}^{(t)}$$

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

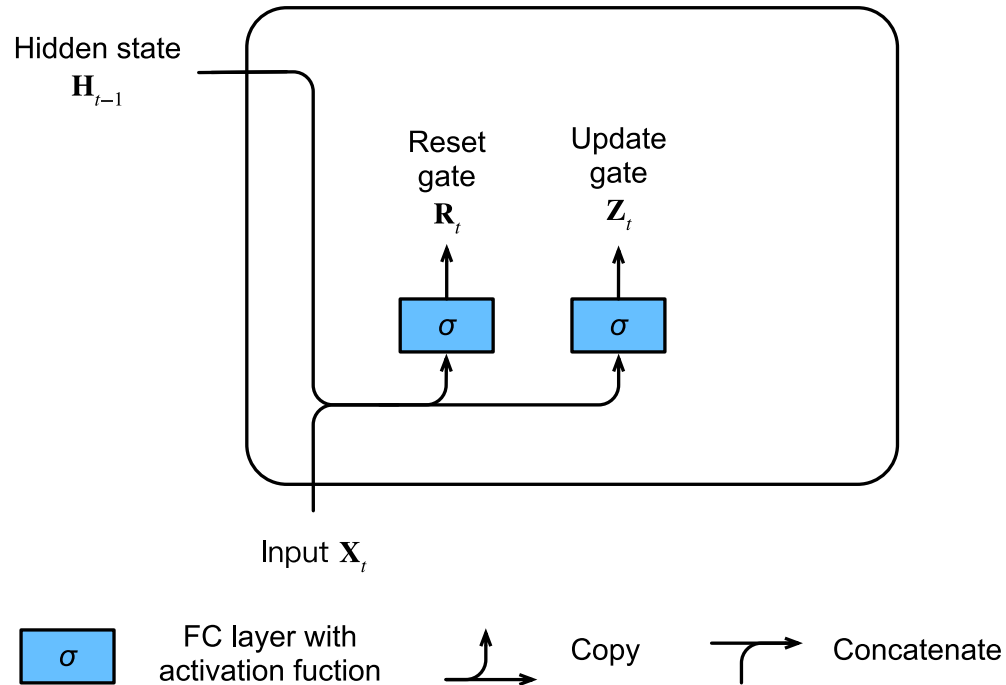
How does this solve vanishing gradient?

Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

Gated Recurrent Units (GRU)



Reset and Update Gates



- Input is $X_t \in \mathbb{R}^{n \times d}$ (number of examples: n , number of inputs: d)
- Hidden state of last timestep $H_{t-1} \in \mathbb{R}^{n \times h}$ (number of hidden states: h).
- Reset gate $R_t \in \mathbb{R}^{n \times h}$ and update gate $Z_t \in \mathbb{R}^{n \times h}$

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

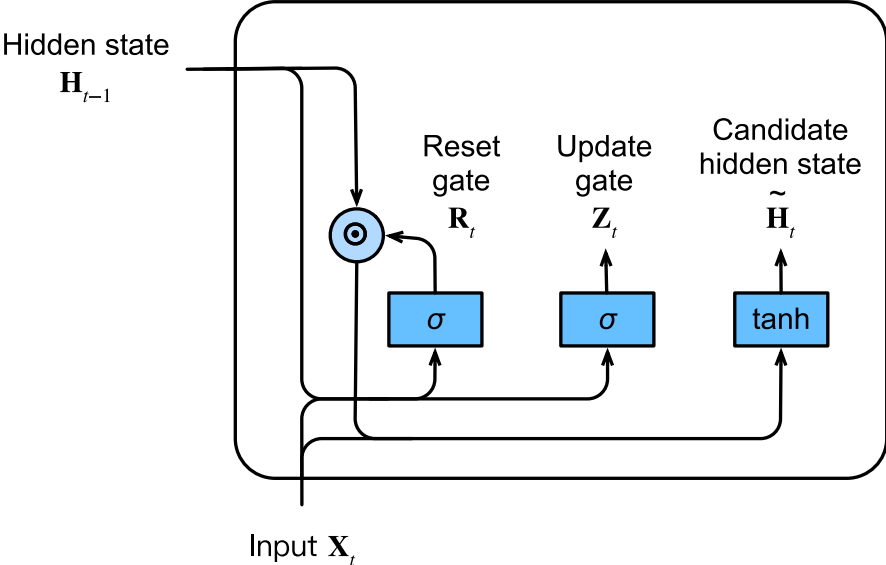
$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

σ is sigmoid activation

$W_{xr}, W_{hr} \in \mathbb{R}^{d \times h}$ and $W_{xz}, W_{hz} \in \mathbb{R}^{h \times h}$ are weight parameters

b_r and $b_z \in \mathbb{R}^{1 \times h}$ are biases

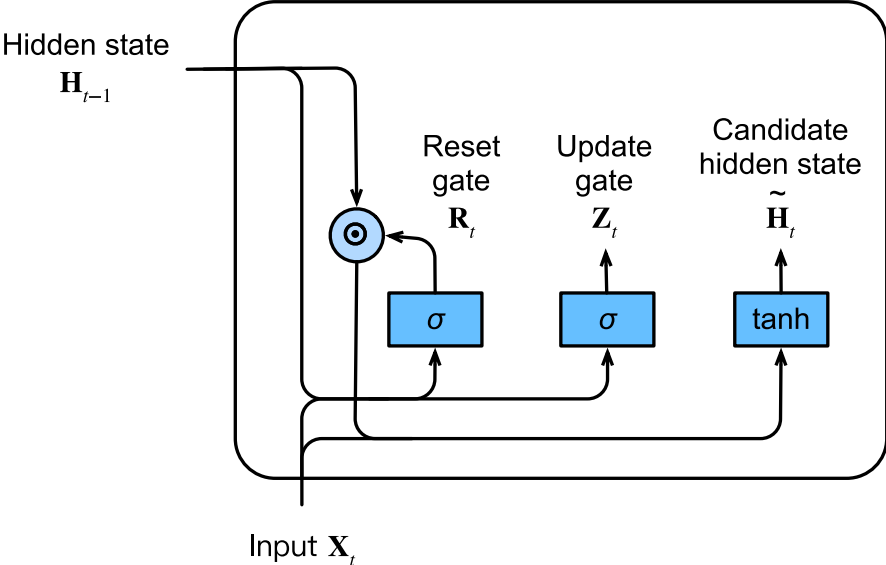
Candidate Hidden State



$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$



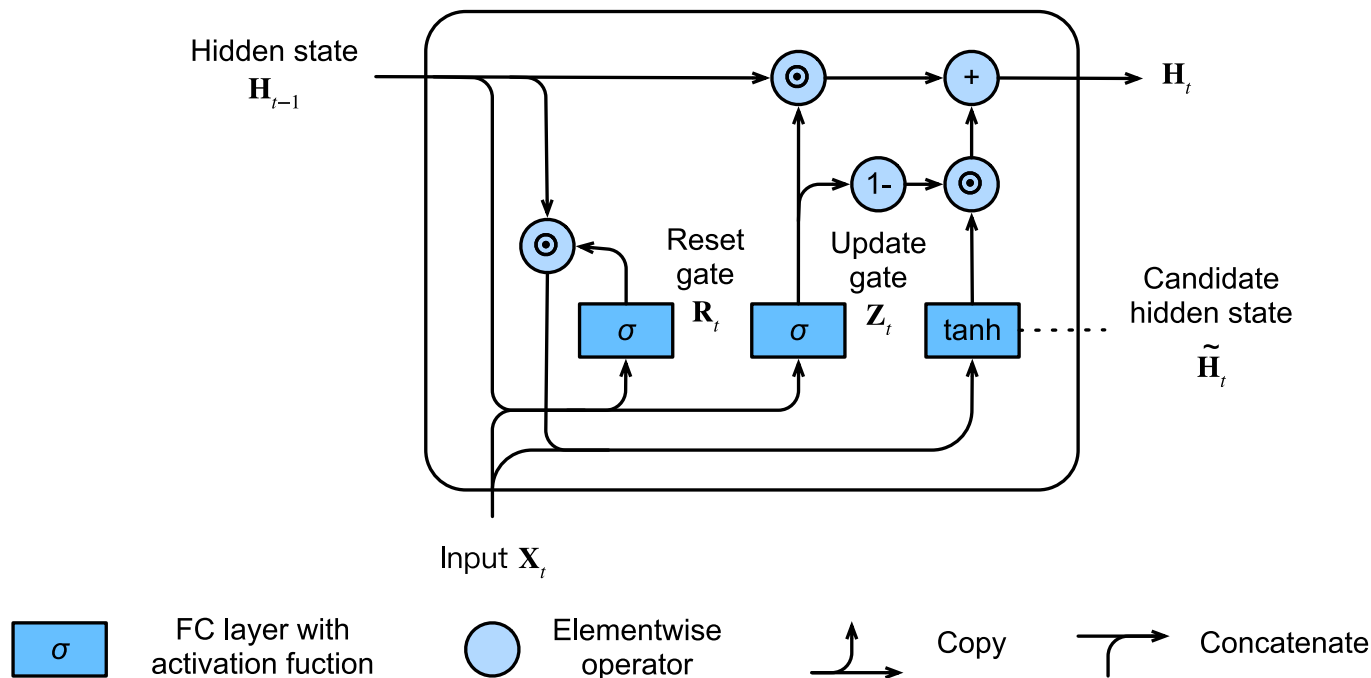
Candidate Hidden State



$$\tilde{H}_t = \tanh(X_t W_{xh} + \underline{(R_t \odot H_{t-1}) W_{hh}} + b_h)$$

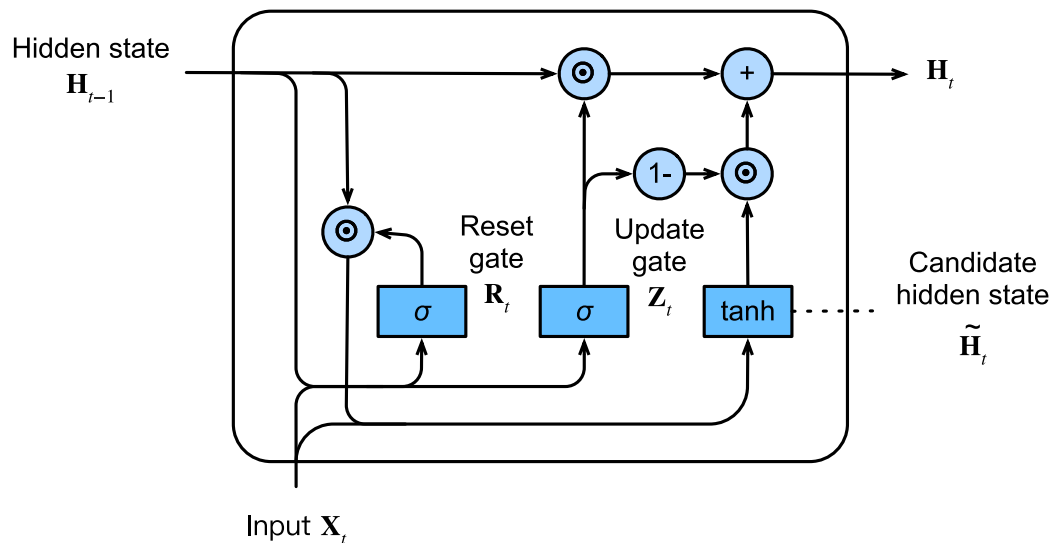


Hidden State



$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

Summary



$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

Long Short-term Memory (LSTM)

- Capable of modeling longer term dependencies by having memory cells and gates that controls the information flow along with the memory cells

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
- On step t , there is a hidden state $\mathbf{h}^{(t)}$ and a cell state $\mathbf{c}^{(t)}$
 - Both are vectors length n
 - The cell stores long-term information
 - The LSTM can erase, write and read information from the cell
- The selection of which information is erased/written/read is controlled by three corresponding gates
 - The gates are also vectors length n
 - The gates are dynamic: their value is computed based on the current context

Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are special RNNs, with different gates and memory cells:

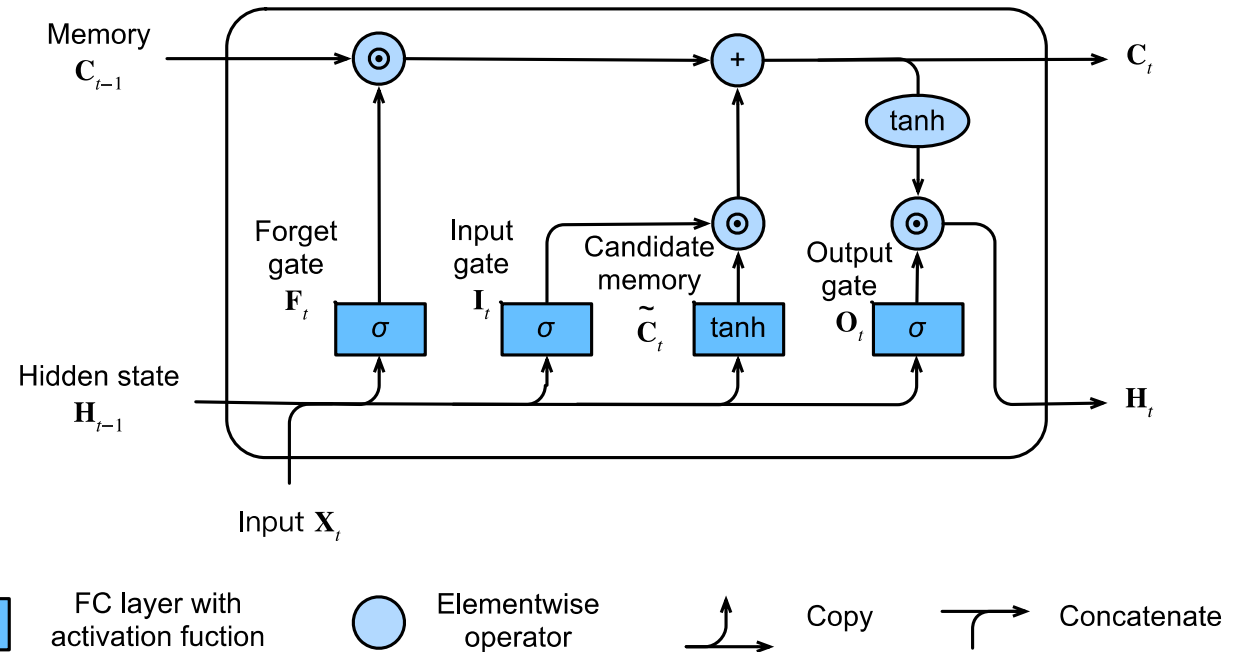
- **Gates:**

- Input gate
- Forget gate
- Output gate

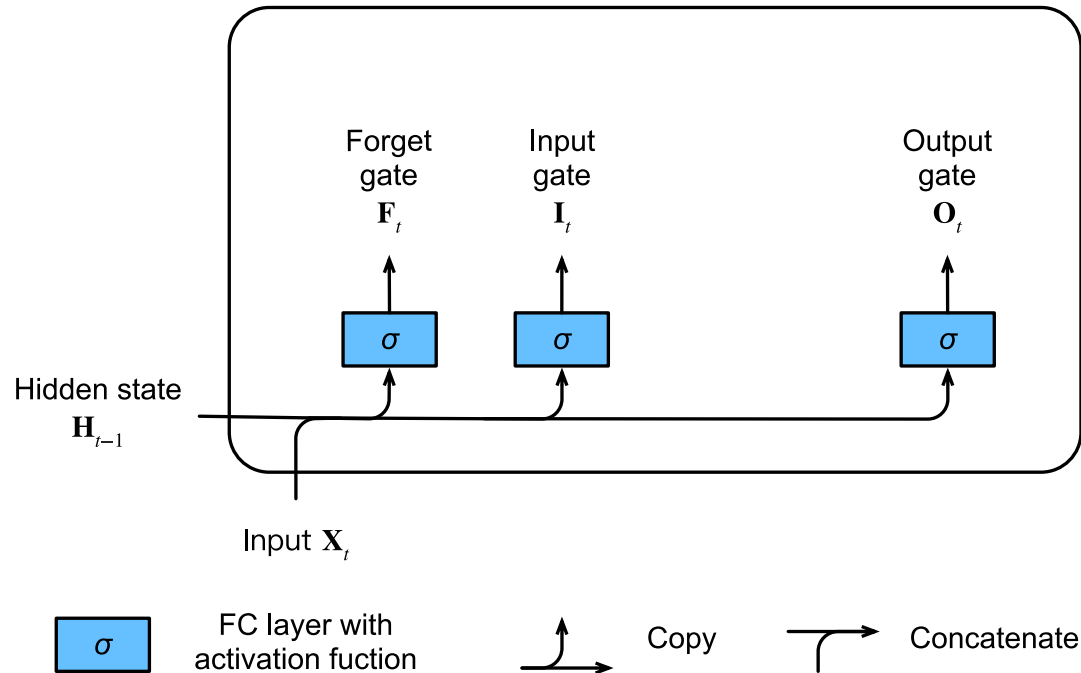
- **Memory cells:**

- Candidate memory cell
- Memory cell

- **Hidden state**

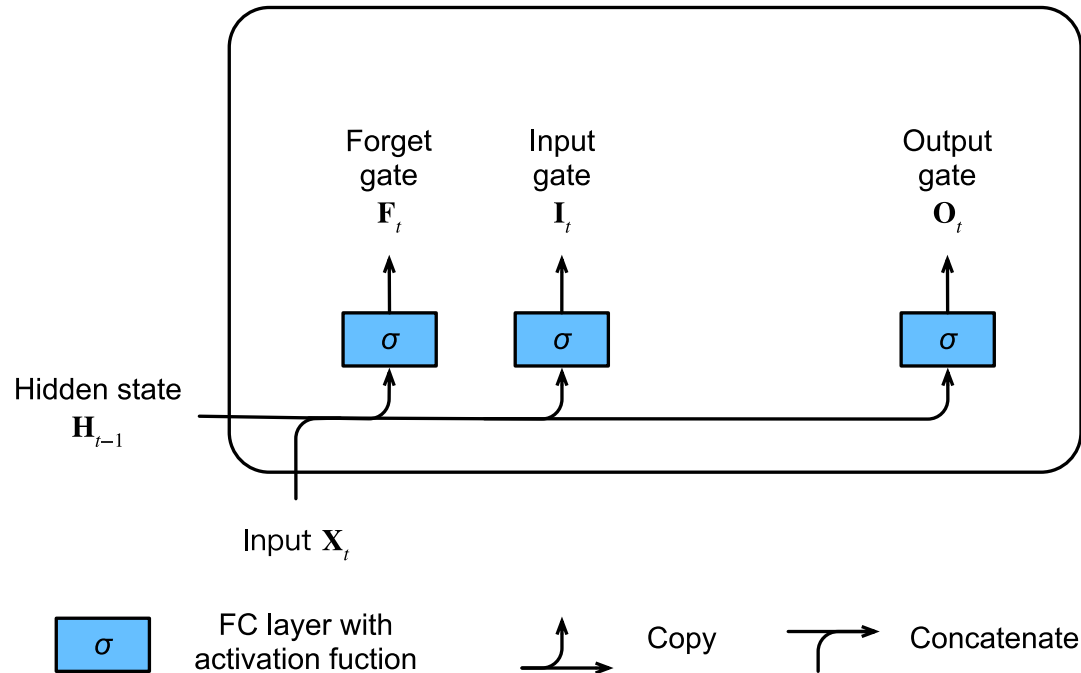


Input, Forget and Output Gates



- Input is $X_t \in \mathbb{R}^{n \times d}$ (number of examples: n , number of inputs: d)
- Hidden state of last timestep $H_{t-1} \in \mathbb{R}^{n \times h}$ (number of hidden states: h).
- All gates use **sigmoid activation function**.

Input, Forget and Output Gates



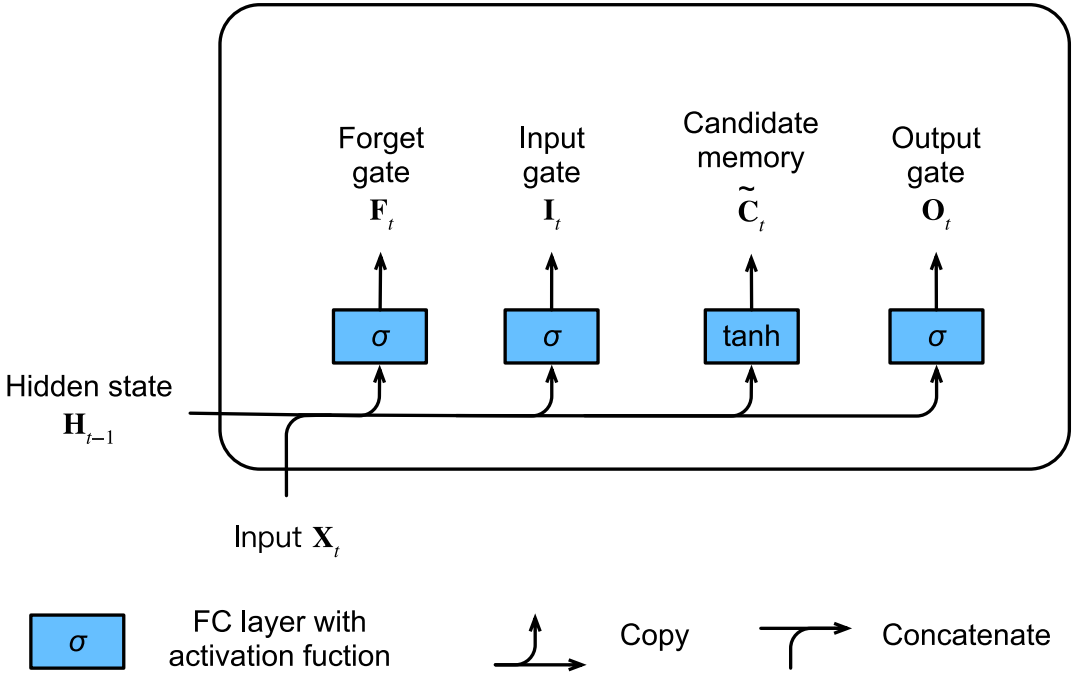
- Input is $X_t \in \mathbb{R}^{n \times d}$ (number of examples: n , number of inputs: d)
- Hidden state of last timestep $H_{t-1} \in \mathbb{R}^{n \times h}$ (number of hidden states: h).
- All gates use **sigmoid activation function**.

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$

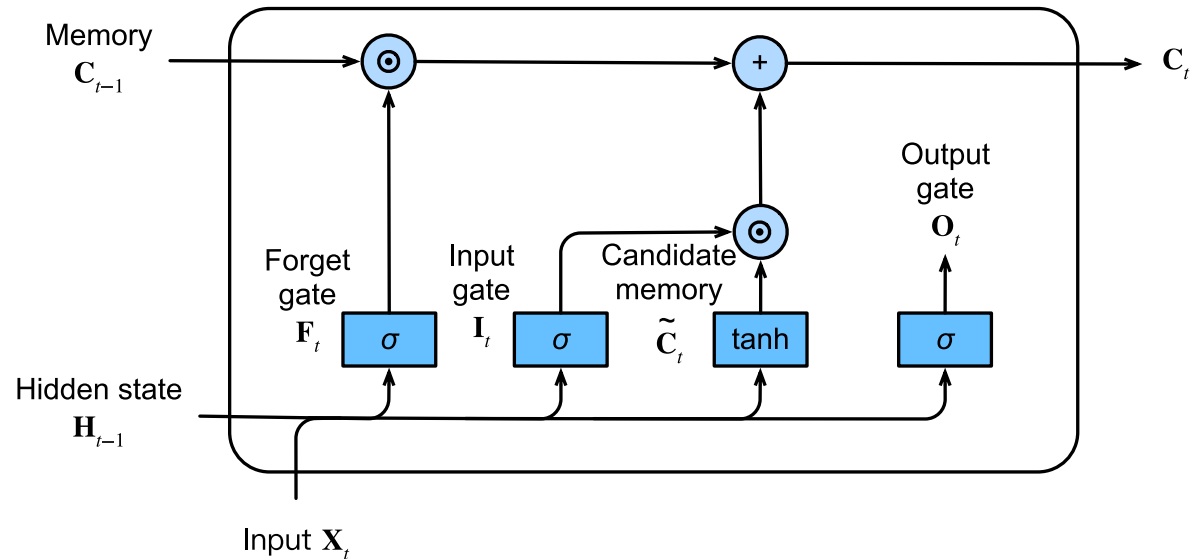
$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$$

Candidate Memory Cell



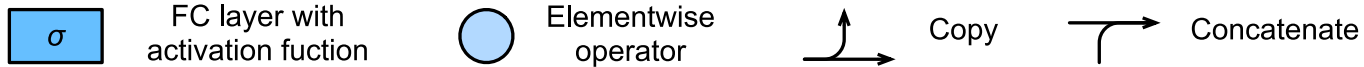
$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

Memory Cell

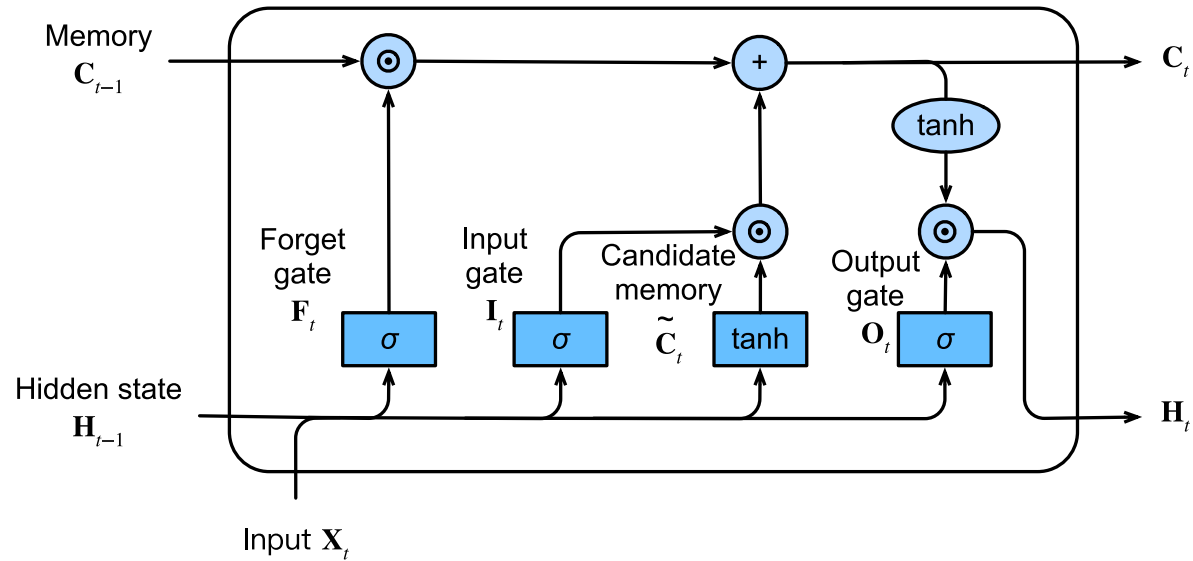


$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

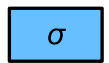
F_t : How much of the old memory will stay and I_t : How much new data will be added



Hidden State



$$H_t = O_t \odot \tanh(C_t)$$



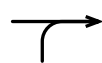
FC layer with
activation fuction



Elementwise
operator

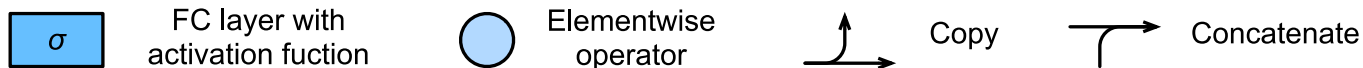
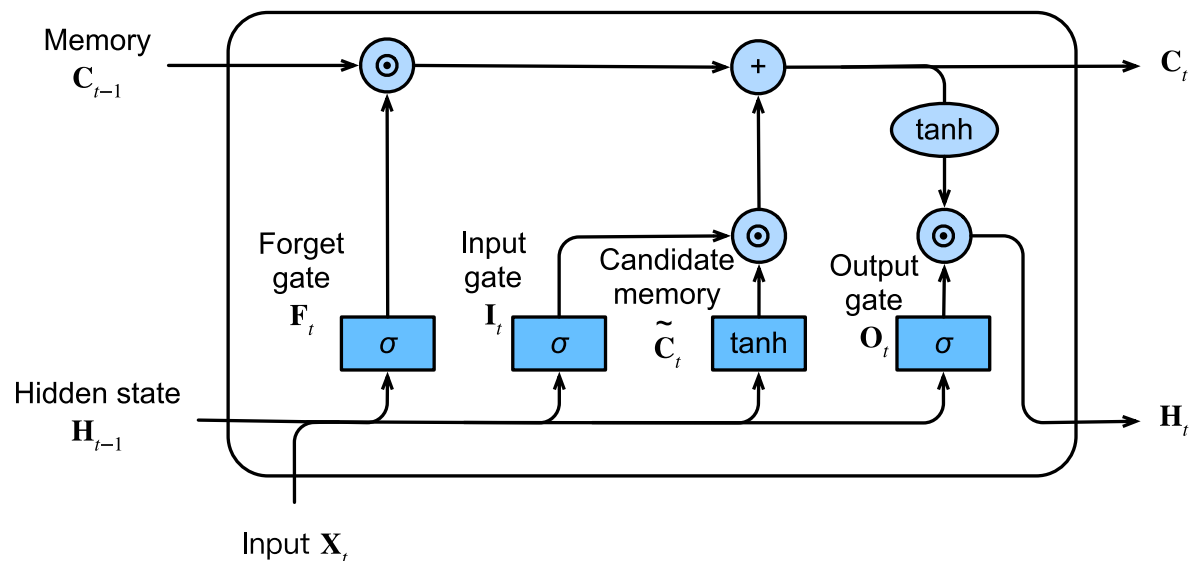


Copy



Concatenate

Summary



$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$$

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

$$H_t = O_t \odot \tanh(C_t)$$

How does LSTM solve vanishing gradients?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
 - e.g. if the forget gate is set to remember everything on every timestep, then the info in the cell is preserved indefinitely
 - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state
- LSTM doesn't *guarantee* that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

Long Short-Term Memory (LSTM)

We have a sequence of inputs $\mathbf{x}^{(t)}$, and we will compute a sequence of hidden states $\mathbf{h}^{(t)}$ and cell states $\mathbf{c}^{(t)}$. On timestep t :

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$\mathbf{f}^{(t)} = \sigma \left(\mathbf{W}_f \mathbf{h}^{(t-1)} + \mathbf{U}_f \mathbf{x}^{(t)} + \mathbf{b}_f \right)$$

$$\mathbf{i}^{(t)} = \sigma \left(\mathbf{W}_i \mathbf{h}^{(t-1)} + \mathbf{U}_i \mathbf{x}^{(t)} + \mathbf{b}_i \right)$$

$$\mathbf{o}^{(t)} = \sigma \left(\mathbf{W}_o \mathbf{h}^{(t-1)} + \mathbf{U}_o \mathbf{x}^{(t)} + \mathbf{b}_o \right)$$

$$\tilde{\mathbf{c}}^{(t)} = \tanh \left(\mathbf{W}_c \mathbf{h}^{(t-1)} + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{b}_c \right)$$

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \circ \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \circ \tilde{\mathbf{c}}^{(t)}$$

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \circ \tanh \mathbf{c}^{(t)}$$

All these are vectors of same length n

Gates are applied using element-wise product

LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used
- The biggest difference is that GRU is quicker to compute and has fewer parameters
- There is no conclusive evidence that one consistently performs better than the other
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data)
- Rule of thumb: start with LSTM, but switch to GRU if you want something more efficient